Precision Unified Automation Framework

Project Documentation

Submitted To

Veeva Systems

Application Under Test

https://automationexercise.com

1 Project Overview

This framework is designed to automate both the User Interface (UI) and API layers of the AutomationExercise web application. It follows industry-standard practices to ensure reliable, maintainable, and scalable test automation coverage across critical user journeys such as user registration, login, and cart management.

The framework adopts a Behaviour Driven Development (BDD) approach using Cucumber, enabling test scenarios to be written in plain English (Gherkin syntax) that are understandable by both technical and non-technical stakeholders. It is structured as a Maven multi-module project, separating UI and API concerns into dedicated modules while sharing common utilities across both

1.1 Project Goals

- Design a unified Hybrid Automation framework covering both UI and REST API testing.
- Implement a clean Maven multi-module architecture with strict separation of concerns.
- Apply the Page Object Model (POM) pattern to isolate UI locators from test logic.
- Implement a layered API design: Client : Request Builder : Validator.
- Support parallel test execution to reduce overall test cycle time.
- Provide structured logging (Log4j2) and reporting (Allure + Cucumber HTML).
- Ensure zero hard-coding: all configuration externalized to properties files.
- Capture automatic screenshots on UI test failure for faster defect detection.

1.2 Scope of Testing

The framework covers two layers of testing — UI and API — targeting the most critical functional areas of the AutomationExercise application.

UI Test Scope

Test Case	Description
TC1 - User Registration	Registers a new user with full account details, verifies account creation, and cleans up by deleting the account.
TC2 - Valid User Login	Logs in with valid credentials and verifies the user is successfully authenticated.
TC3 - Invalid User Login	Attempts login with an invalid email and verifies the appropriate error message is displayed.
TC4 - Add Product to Cart	Navigates to the products page, adds two products to the cart, and validates product names, quantities, and total prices.
TC5 - Remove Product from Cart	Adds a product to the cart, navigates to the cart page, removes the product, and verifies it no longer appears in the cart.

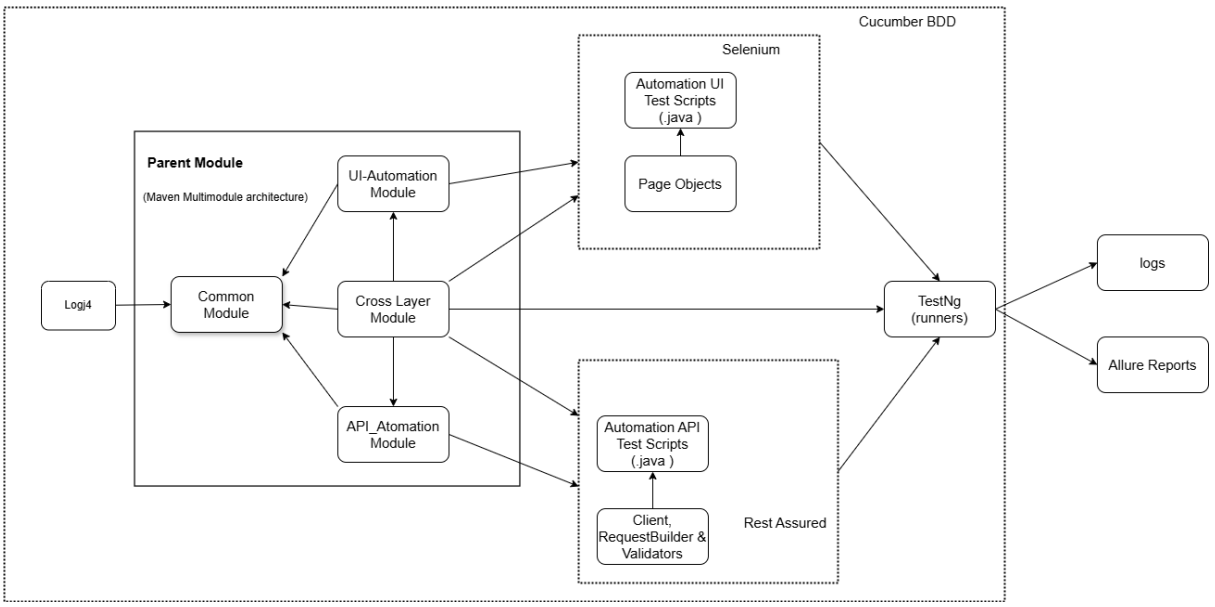
API Test Scope

Test Case	Description
TC1 - Get All Products	Sends a GET request to the products list endpoint, validates the response code, and verifies the response matches the defined JSON schema.
TC2 - Search Product	Sends a POST request to search for a product by name, validates the response code, verifies the returned product details, and confirms schema compliance.
TC3 - Create User Account	Sends a POST request to register a new user account and validates the response code and success message.
TC4 - Delete User Account	Sends a DELETE request to delete existing user account and validates the response code.
TC5 - Update User Account	Sends a PUT request to update an existing user account and validates the response code, verify where the data is updated.
TC6 – Negative Validation	Sends a DELETE request to delete user account that is present and validates whether is throw expected error or not

Out of Scope

- Placing Order, Payment and checkout flows
- Subscription functionality
- Non-functional testing such as performance, security, or accessibility testing

2 Architecture Diagram



Parent Module (pom.xml) : Manages dependencies, plugins, and build configuration for all modules.

Common Module : Provides shared utilities, configurations, and logging used by UI and API modules.

UI Automation Module : Handles frontend web application testing using Selenium.

Page Objects : Stores UI locators and reusable page actions following the Page Object Model design pattern.

Automation UI Test Scripts : Implements UI test scenarios by interacting with Page Objects.

API Automation Module : Performs backend API testing using Rest Assured.

API Client : Responsible for sending HTTP requests to API endpoints.

Request Builder : Constructs API requests with headers, payloads, and parameters.

Validators : Validates API responses such as status codes and response data.

Automation API Test Scripts : Contains implementation of API test cases.

TestNG Runners : Executes UI and API test suites and controls test execution flow.

Cucumber BDD : Defines test scenarios in Gherkin format for behavior-driven testing.

Log4j Logging : Captures execution logs for debugging and monitoring test runs.

Allure Reports : Generates detailed HTML reports from test execution results.

3 Module Breakdown

The framework is structured as a maven multi-module project under the parent artifact automationexercise-hybrid-framework. It consists of three modules, each with a clearly defined responsibility, promoting separation of concerns and reusability across the project.

3.1 Parent Module — automationexercise-hybrid-framework

The root pom.xml acts as the parent POM for all child modules. It is responsible for:

- Declaring all three child modules: common, ui-automation, and api-automation.
- Centralizing all dependency versions through <dependencyManagement>, ensuring consistent library versions are used across modules without repeating version numbers in child POMs.
- Defining shared project properties such as Java version, Selenium, Cucumber, TestNG, RestAssured, Allure, Log4j2, and JavaFaker versions.

3.2 Module 1 — common

Purpose: Provides shared utilities that can be consumed by both the ui-automation and api-automation modules, avoiding code duplication.

Package: logger

Class	Description
LogsFactory	A utility class that wraps Apache Log4j2's **LogManager**. Exposes a static getLogger() method used throughout both modules to obtain named loggers for consistent, structured logging.

This module is compiled as a regular JAR and referenced as a dependency by the other two modules.

3.3 Module 2 — ui-automation

Purpose: Contains all components required to automate and execute UI test scenarios against the AutomationExercise web application using Selenium WebDriver and Cucumber.

Source root: src/test/java

Sub-packages and their responsibilities:

driver

Class	Description
DriverFactory	Manages WebDriver lifecycle using a ThreadLocal instance to support parallel execution. Handles driver initialization for Chrome and Edge, applies implicit waits, and provides getDriver() and quitDriver() methods.

Class	Description
OptionsFactory	Builds browser-specific options for Chrome, Edge, and Firefox. Applies configurations such as disabling notifications, suppressing automation flags, blocking third-party cookies, and mapping ad-serving domains to 0.0.0.0 to prevent ads from interfering with tests.

Config

Class	Description
ConfigReader	Loads config.properties at class initialization using a static block. Provides a static getProperty(String key) method that throws a RuntimeException if a requested key is not found, ensuring misconfiguration is caught early.

Hooks

Class	Description
Hooks	Cucumber @Before hook initializes the WebDriver and navigates to the base URL before each scenario. The @After hook captures a screenshot and attaches it to the Cucumber report if the scenario fails, then quits the driver.

Pages

Class	Description
HomePage	Locators and actions for the home page — clicking Signup/Login, Products, Cart buttons, and verifying the logged-in username.
LoginPage	Handles both the login form and signup name/email form present on the same page. Provides methods to enter credentials, click buttons, and retrieve displayed messages.
SignupPage	Fills in the full account registration form including title, password, date of birth, address fields, and country dropdown using Selenium's Select class. Uses JavaScript executor to handle the Create Account button click reliably.
AccountCreatedPage	Retrieves the account creation confirmation message and provides the Continue button action.
AccountDeletedPage	Retrieves the account deletion confirmation message and provides the Continue button action.
ProductPage	Manages product interactions on the products listing page — adding the first and second products to cart, clicking Continue Shopping, and navigating to the cart. Also stores product names for later assertion and calculates total price validation.

Class	Description
CartPage	Handles cart page interactions — verifying the cart table is displayed, retrieving product names, clicking the delete button, waiting for product removal, and checking product visibility.

stepdefinitions

Class	Description
RegisterSteps	Step definitions covering the full user registration and deletion flow.
LoginSteps	Step definitions for valid and invalid login scenarios.
AddProductSteps	Step definitions for adding two products to cart and validating names, quantities, and prices.
RemoveProductSteps	Step definitions for navigating to cart, removing a product, and verifying removal.

utils

Class	Description
WaitUtil	Creates and returns a WebDriverWait instance using the current thread's driver from DriverFactory.
ScreenshotUtil	Captures a screenshot as a byte array using Selenium's TakesScreenshot interface, used by the Hooks class on failure.
RandomUtil	Generates a unique email address by appending the current timestamp to the base email from config, ensuring no duplicate registrations during test runs.
RegisterDataUtil	Builds and returns a Map<String, String> of all registration form fields by reading values from config.properties, with a dynamically generated password.

runners

Class	Description
TestRunner	Default sequential test runner. Picks up all feature files from src/test/resources/features and generates Allure, HTML, JSON, and rerun reports.
ParallelRunner	Extends TestRunner configuration and overrides the scenarios() @DataProvider with parallel = true to enable concurrent scenario execution.

Class	Description
FailedTestRunner	Reads from target/failed_scenarios.txt generated by a previous run and re-executes only the failed scenarios.

resources

File	Description
config.properties	Holds base URL, browser setting, registration data, and login credentials.
features/	Contains all Gherkin feature files: register.feature, login.feature, addProduct.feature, removeProduct.feature.
log4j2.xml	Log4j2 configuration for console and file-based logging.
allure.properties	Allure results directory configuration.

3.4 Module 3 – api-automation

Purpose: Contains all components required to automate and validate REST API endpoints of the AutomationExercise application using RestAssured and Cucumber.

Source root: src/test/java

Sub-packages and their responsibilities:

constants

Class	Description
ApiEndpoints	Defines all API endpoint constants — base URL, products list endpoint, search product endpoint, and create account endpoint — in a single place for easy maintenance.

config

Class	Description
ConfigReader	Identical in purpose to the UI module's ConfigReader. Loads config.properties and provides safe property retrieval.

builder

Class	Description
RequestBuilder	Central class for building and sending HTTP requests using RestAssured. Provides get() for GET requests, post() for form-param POST requests (product search), and postCreateAccount() for multi-param user registration POST requests. Internally converts a User POJO to a form parameter map.

clients

Class	Description
ProductApiClient	Wraps RequestBuilder to expose product-specific API calls — getAllProducts() and getSingleProduct(String parameter).
UserAccountApiClient	Wraps RequestBuilder to expose user account API calls — registerUser(User user).

pojo

Class	Description
User	Plain Java object representing a user with all registration fields — name, email, password, title, date of birth, address details, and mobile number — with full getters and setters.

util

Class	Description
UserDataBuilder	Constructs a fully populated User object by reading values from config.properties. Generates a unique email using a timestamp to avoid conflicts across test runs.

validators

Class	Description
CommonValidator	Base validator class providing reusable assertion methods: validateStatusCode(), validateMessage(), validateResponseTime(), validateContentType(), and validateHeader(). Note: validateStatusCode() reads the status code from the JSON response body field responseCode, not the HTTP status code.
ProductValidator	Extends CommonValidator. Adds validateProductSchema() for JSON schema validation against products-schema.json, and validateSearchedProductName() to assert the returned product category matches the search term.
UserAccountValidator	Extends CommonValidator. Currently inherits all base validation methods without additional overrides.

stepdefinitions

Class	Description
ProductSteps	Step definitions for getting all products and searching a product by name, wiring Cucumber steps to ProductApiClient and ProductValidator.
UserAccountSteps	Step definitions for registering a new user account, wiring Cucumber steps to UserAccountApiClient and UserAccountValidator.

runners

Class	Description
ProductRunner	Single Cucumber-TestNG runner that picks up all API feature files and generates Allure, HTML, and JSON reports.

resources

File	Description
config.properties	Holds registration and login data used by API tests.
features/	Contains product.feature and userAccount.feature.
schemas/products-schema.json	JSON schema definition used to validate the structure of the products API response, covering responseCode, products array, and each product's required fields.
allure.properties	Allure results directory configuration.

3.5 Module 4 — cross-layer-integration

Purpose: Provides cross-layer hybrid test scenarios that combine API and UI automation within a single end-to-end test flow. This module depends on both ui-automation and api-automation modules, orchestrating them together to validate that actions performed at the API layer are correctly reflected and usable at the UI layer.

Package root: com.automationexercise.crosslayer

Source root: src/test/java

Sub-packages and their responsibilities:

hooks

Class	Description
Hooks	Cucumber @Before hook initializes the WebDriver by delegating to the ui-automation module's DriverFactory and navigates to the base URL read from ConfigReader. The @After hook captures a screenshot via ScreenshotUtil if the scenario fails and attaches

Class	Description
	it to the Cucumber report, then quits the driver. Reuses infrastructure directly from the ui-automation module.

stepdefinitions

Class	Description
HybridSteps	The central step definitions class for cross-layer scenarios. It holds references to both API-layer components (UserApiClient, UserDataBuilder, UserValidator) and UI-layer components (DriverFactory, HomePage, LoginPage, AccountCreatedPage, AccountDeletedPage). The API steps register a user and validate the response, then the UI steps use the dynamically generated credentials from that same User object to log in through the browser — creating a direct, stateful handoff between the two layers.

runners

Class	Description
CrossLayerTestRunner	Primary Cucumber-TestNG runner that picks up all feature files from src/test/resources/features. Glue is scoped to the crosslayer stepdefinitions and hooks packages. Generates Allure, HTML, JSON, and rerun reports.
FailedTestRunner	Reads from target/failed_scenarios.txt produced by a previous run and re-executes only the failed cross-layer scenarios.

Source root: src/test/resources

Sub-packages and their responsibilities:

File	Description
features/hybrid.feature	Contains the cross-layer Gherkin scenario that first registers a user via API, validates the API response and schema, then logs in with the same credentials through the browser UI, verifies successful authentication, and cleans up by deleting the account.
schemas/user-schema.json	JSON schema used to validate the structure of the user registration API response, asserting that both responseCode (number) and message (string) fields are present.

4 Design Patterns Used

4.1 Page Object Model (POM)

Applied in: ui-automation : pages package

Page Object Model is the primary structural pattern used in the UI automation module. Each page of the application is represented as a dedicated Java class that encapsulates all the web element locators and the actions that can be performed on that page.

How it is applied:

- All By locators are declared as private fields within the page class.
- Public methods expose meaningful actions such as clickSignupLogin(), fillAccountDetails(), or removeProductFromCart() rather than exposing raw element interactions.
- Step definition classes interact only with page object methods, never with Selenium WebDriver or By locators directly.

Benefit: Any change in the UI — such as a locator update — requires a change in only one place (the page class), without touching the test logic in step definitions.

4.2 Factory Pattern

Applied in: ui-automation : driver.DriverFactory and driver.OptionsFactory

The Factory pattern is used to abstract the creation of objects whose exact type is determined at runtime based on input parameters.

How it is applied:

- DriverFactory.initDriver(String browser) uses a switch statement on the browser name to instantiate either a ChromeDriver or EdgeDriver, shielding the rest of the framework from knowing which concrete driver is being created.
- OptionsFactory provides dedicated static factory methods — buildChromeOptions(), buildEdgeOptions(), and buildFirefoxOptions() — each returning a fully configured browser options object.

Benefit: Adding support for a new browser requires changes only within DriverFactory and OptionsFactory, with no impact on any other class in the framework.

4.3 Singleton Pattern

Applied in: ui-automation : driver.DriverFactory and config.ConfigReader (both modules)

How it is applied:

- DriverFactory uses a ThreadLocal<WebDriver> to maintain a single WebDriver instance per thread. This ensures each test thread has exactly one driver throughout its lifecycle, preventing conflicts during parallel execution.

- ConfigReader in both modules uses a static properties block that loads the config.properties file once at class initialization. The same Properties object is shared across all calls to getProperty() throughout the entire test run.

Benefit: Prevents redundant object creation, ensures consistent state, and in the case of DriverFactory, enables thread-safe driver management during parallel test execution.

4.4 Builder Pattern

Applied in: api-automation : builder.RequestBuilder

The Builder pattern is used to construct complex RestAssured request specifications in a step-by-step, readable manner.

How it is applied:

- RequestBuilder.buildRequest() creates a base RequestSpecification with the base URI.
- The get(), post(), and postCreateAccount() methods build upon this base specification by chaining additional configurations such as content type, form parameters, and response extraction.
- The userToFormParams() private method converts a User POJO into a flat Map<String, String> of form parameters, which are then applied to the request specification iteratively.

Benefit: Centralizes all request construction logic, making it easy to modify how requests are built without affecting API client classes or step definitions.

4.5 Layered Architecture

Applied in: api-automation : client.APIClient, builder.RequestBuilder, validator.ResponseValidator

The Layered Architecture pattern organizes the automation framework into separate logical layers, each handling a distinct responsibility such as request construction, execution, and validation.

How it is applied:

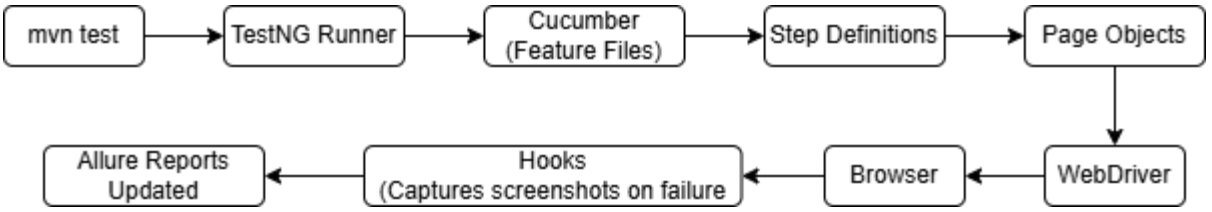
- The RequestBuilder layer is responsible for constructing API request specifications, including payloads and parameters.
- The APIClient layer executes HTTP requests using the specifications provided by the RequestBuilder.
- The Validator layer verifies API responses by performing assertions on status codes, response bodies, and headers.
- Each layer has a single responsibility and interacts with other layers in a structured manner to complete the API testing workflow.

Benefit: Improves modularity and maintainability by separating request creation, execution, and validation logic. This makes the framework easier to extend, debug, and reuse across different API test scenarios.

5 Framework Flow Diagrams

Two flow diagrams — one for UI, one for API. Show step by step what happens from Maven command to report generation.

5.1 UI Flow:



Maven (mvn test) – Acts as the entry point of execution, triggering the test lifecycle, compiling code, and initiating the automation suite through configured plugins and runners.

• **TestNG Runner** – Serves as the execution engine that integrates with Cucumber, manages test execution flow, supports parallel execution, and controls configurations like test suites and listeners.

• **Cucumber (Feature Files)** – Contains test scenarios written in Gherkin language (Given-When-Then), enabling clear, business-readable definitions of application behavior.

• **Step Definitions** – Maps each Gherkin step to Java methods, acting as a bridge between human-readable scenarios and automation logic while orchestrating calls to page objects.

• **Page Objects (POM Layer)** – Encapsulates UI elements (locators) and reusable actions (methods), ensuring separation of UI interaction logic from test logic and improving maintainability.

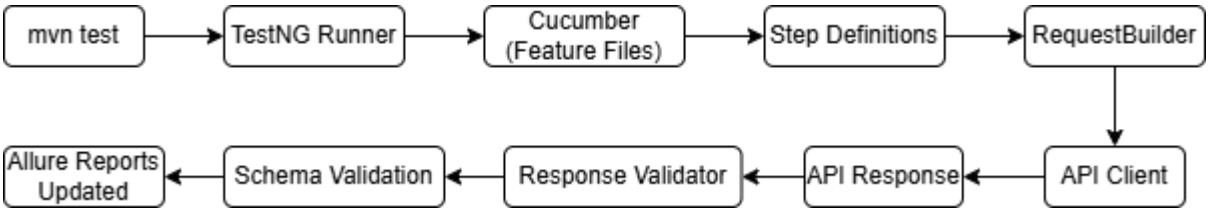
• **WebDriver Layer** – Provides an interface to interact with browsers, executing commands like click, sendKeys, navigation, and synchronization using Selenium WebDriver APIs.

• **Browser Execution** – Represents the actual browser instance (Chrome, Edge, etc.) where test steps are performed, simulating real user interactions with the application.

• **Hooks (Failure Handling & Setup)** – Implements pre- and post-execution logic such as test setup, teardown, and capturing screenshots on failure for debugging purposes.

• **Allure Reports** – Collects execution results, logs, and screenshots to generate detailed, visually rich reports that help analyze test outcomes and failures.

5.2 API Flow:



• **Maven (mvn test)** – Initiates the API test execution lifecycle, ensuring build, dependency resolution, and triggering of the test framework.

• **TestNG Runner** – Controls execution of Cucumber-based API tests, manages parallel runs, and integrates reporting and configuration settings.

• **Cucumber (Feature Files)** – Defines API test scenarios in Gherkin syntax, describing endpoints, request actions, and expected responses in a readable format.

• **Step Definitions** – Converts feature steps into executable code, coordinating interactions between RequestBuilder, APIClient, and validation layers.

• **RequestBuilder (Builder Layer)** – Constructs API requests in a modular and reusable manner by setting base URI, headers, query parameters, and payloads step-by-step.

• **API Client (Execution Layer)** – Handles actual HTTP operations (GET, POST, PUT, DELETE) using RestAssured, executing requests built by RequestBuilder and returning responses.

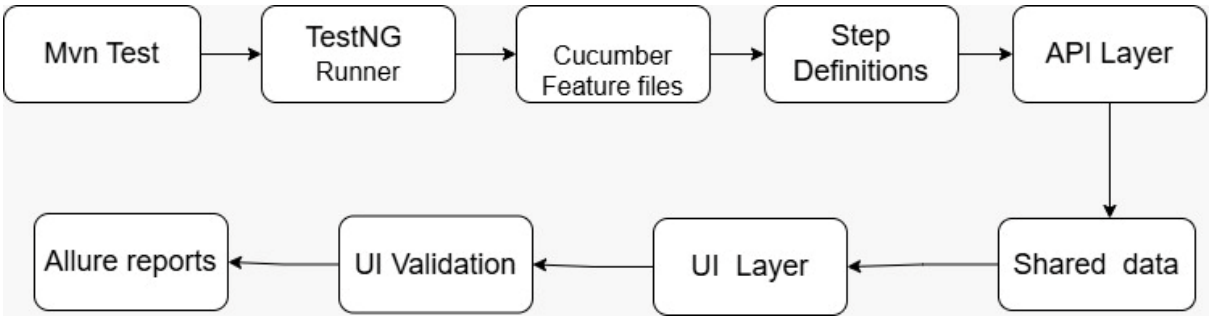
• **API Response Handling** – Captures the complete HTTP response including status code, headers, and body, which is then passed to validation layers for verification.

• **Response Validator** – Performs assertions on API responses such as validating status codes, response body values, and headers to ensure correctness of API behavior.

• **Schema Validation** – Verifies that the API response structure matches the expected JSON schema, ensuring contract consistency and preventing structural regressions.

• **Allure Reports** – Aggregates request/response logs, validation results, and execution status into detailed reports for analysis and debugging.

5.3 Cross layer Flow



- **Maven Test (mvn test)** → Entry point that triggers the complete automation execution pipeline.
- **TestNG Runner** → Controls and orchestrates execution of Cucumber scenarios for both UI and API tests.
- **Cucumber Feature Files** → Define business-readable test scenarios that unify UI and API automation under BDD.
- **Step Definitions** → Act as the integration layer mapping feature steps to either UI or API actions.

- **API Layer** → Performs backend operations (requests, responses, validations) and generates dynamic test data.
- **Shared Data** → Stores and transfers data (like tokens, IDs) between API and UI layers for seamless integration.
- **UI Layer** → Executes frontend actions using data prepared by API, enabling end-to-end validation.
- **UI Validation** → Verifies that UI behavior correctly reflects backend (API) operations.
- **Allure Reports** → Provides a unified report combining both UI and API test results with detailed insights.

6. Folder Structure

The project follows a Maven multi-module architecture with four modules:

- common → shared utilities
- ui-automation → UI testing (Selenium + Cucumber)
- api-automation → API testing (RestAssured + Cucumber)
- cross-layer-integration → Hybrid UI + API validation

6.1 Root Level

```
automationexercise-hybrid-framework/
│
├── pom.xml                  ← Parent POM
├── common/
├── ui-automation/
├── api-automation/
└── cross-layer-integration/
```

6.2 common Module

Provides shared utilities like configuration and logging.

```
common/
├── src/main/java
│   ├── config
│   │   └── ConfigReader.java
│   └── logger
│       └── LogsFactory.java
│
├── src/main/resources
│   └── config.properties
│
└── pom.xml
```

6.3 ui-automation Module

Handles UI automation using Selenium + Cucumber + TestNG.

```
ui-automation/
├── src/main/java
│   ├── com.automationexercise.ui.driver
│   │   ├── DriverFactory.java
│   │   └── OptionsFactory.java
│   │
│   ├── com.automationexercise.ui.pages
│   │   ├── AccountCreatedPage.java
```

```
│   │   ├── AccountDeletedPage.java
│   │   ├── CartPage.java
│   │   ├── HomePage.java
│   │   ├── LoginPage.java
│   │   ├── ProductPage.java
│   │   └── SignupPage.java
│   │
│   └── com.automationexercise.ui.utils
│       ├── RandomUtil.java
│       ├── RegisterDataUtil.java
│       ├── ScreenshotUtil.java
│       └── WaitUtil.java
│
├── src/test/java
│   ├── com.automationexercise.ui.hooks
│   │   └── Hooks.java
│   │
│   ├── com.automationexercise.ui.runners
│   │   ├── UITestRunner.java
│   │   ├── ParallelTestRunner.java
│   │   └── FailedTestRunner.java
│   │
│   └── com.automationexercise.ui.stepdefinitions
│       ├── AddProductSteps.java
│       ├── LoginSteps.java
│       ├── RegisterSteps.java
│       └── RemoveProductSteps.java
│
├── src/test/resources
│   ├── features
│   │   ├── f1_register.feature
│   │   ├── f2_login.feature
│   │   └── f3_cart.feature
│   ├── allure.properties
│   └── log4j2.xml
│
├── master-testng.xml
├── parallel-testng.xml
└── pom.xml
```

6.4 api-automation Module

Handles API automation using RestAssured with layered architecture.

```
api-automation/
├── src/main/java
│   ├── com.automationexercise.api.builder
│   │   └── RequestBuilder.java
│   │
│   ├── com.automationexercise.api.clients
│   │   ├── ProductApiClient.java
│   │   └── UserApiClient.java
│   │
│   ├── com.automationexercise.api.constants
│   │   └── ApiEndpoints.java
│   │
│   ├── com.automationexercise.api.payloads
│   │   └── User.java
│   │
│   ├── com.automationexercise.api.util
│   │   └── UserDataBuilder.java
│   │   └── UserFactory.java
│   │
│   └── com.automationexercise.api.validators
│       ├── CommonValidator.java
│       ├── ProductValidator.java
│       └── UserValidator.java
│
├── src/test/java
│   ├── com.automationexercise.api.hooks
│   │   ├── Hooks.java
│   ├── com.automationexercise.api.runners
│   │   ├── APITestRunner.java
│   │   ├── ParallelTestRunner.java
│   │   └── FailedTestRunner.java
│   │
│   └── com.automationexercise.api.stepdefinitions
│       ├── ProductSteps.java
│       ├── UserAccountSteps.java
│       ├── DeleteUserAccountSteps.java
│       └── UpdateAccountSteps.java
│
├── src/test/resources
```

```
│   ├── features
│   │   ├── f1_product.feature
│   │   ├── f2_user.feature
│   │   └── f3_negative_validation.feature
│   │
│   ├── schemas
│   │   ├── products-schema.json
│   │   └── user-schema.json
│   │
│   └── allure.properties
│
├── master-testng.xml
├── parallel-testng.xml
└── pom.xml
```

6.5 cross-layer-integration Module

Handles hybrid scenarios combining UI + API validations.

```
cross-layer-integration/
├── src/test/java
│   ├── com.automationexercise.crosslayer.hooks
│   │   └── Hooks.java
│   │
│   ├── com.automationexercise.crosslayer.runners
│   │   ├── CrossLayerTestRunner.java
│   │   └── FailedTestRunner.java
│   │
│   └── com.automationexercise.crosslayer.stepdefinitions
│       └── HybridSteps.java
│
├── src/test/resources
│   ├── features
│   │   └── hybrid.feature
│   │
│   └── schemas
│       └── user-schema.json
│
├── allure-results
├── target
├── test-output
└── pom.xml
```

7. Configuration Management

The framework follows a configuration-driven approach where all environment-specific values—such as base URLs, browser settings, credentials, and test data—are stored in external configuration files instead of being hardcoded in the code.

This allows easy switching between environments (Dev, QA, Prod) without modifying Java classes or feature files.

Configuration Principle:

No URL, browser name, or credential is written directly in code. All values are read dynamically from configuration files or constants, ensuring better maintainability and flexibility.

7.1 Common Module — config.properties

Location:
common/src/main/resources/config.properties

This file acts as a central configuration file for the framework.

Contains:

- base.url → Application URL
- browser → Browser type (Chrome / Edge)
- Registration data (name, email, password, DOB, address, etc.)
- Login credentials (valid & invalid)

Benefits:

- Centralized configuration
- Easy updates
- No code changes required

7.2 ConfigReader Implementation

The ConfigReader class (in common module) loads the properties file once using a static block and provides values using a getProperty() method.

- Pattern Used:
- Singleton Pattern
- Advantages:
- Avoids repeated file loading
- Ensures consistent data across modules

7.3 API Endpoints Constants

The ApiEndpoints class in the API module stores all API URLs as constants.

- Example:

BASE_URL = https://automationexercise.com/api
GET_PRODUCTS_URL = /productsList
SEARCH_PRODUCT_URL = /searchProduct
REGISTER_USER_URL = /createAccount

- Benefit:
- Single place to update API paths
- Cleaner and maintainable code

7.4 Dynamic Data Generation

To avoid duplicate data issues, the framework generates unique email IDs during execution.

- Used In:
- RandomUtil (UI)
- UserDataBuilder (API)
- Benefit:
- Prevents duplicate user errors
- Improves test reliability

7.5 Summary

- Configuration stored in common module
- Accessed using ConfigReader
- API URLs managed via constants
- Test data generated dynamically

8. Parallel Execution Strategy

The framework is designed to support parallel test execution at the scenario level. Three runners have been implemented in the ui-automation module to provide complete flexibility across sequential execution, parallel execution, and failed test re-runs.

> Thread Safety Guarantee: Parallel execution is safe because DriverFactory.initDriver() stores the WebDriver in a ThreadLocal<WebDriver>. Each thread accesses its own isolated driver instance via DriverFactory.getDriver(), with zero shared mutable state between threads.

8.1 TestRunner — Sequential Execution

TestRunner extends AbstractTestNGCucumberTests without overriding the scenarios() DataProvider. All Cucumber scenarios execute sequentially in a single thread. This runner is used during:

- Initial framework setup and debugging.
- CI baseline runs where test order dependency exists.
- Environments with limited browser licensing or resource constraints.

8.2 ParallelRunner — Parallel Execution

ParallelRunner also extends AbstractTestNGCucumberTests and overrides the scenarios() DataProvider with the @DataProvider(parallel = true) annotation. This instructs TestNG to supply each Cucumber scenario to a separate thread from its thread pool, enabling multiple browser sessions to run concurrently. The thread pool size is controlled by the TestNG XML configuration (testng-parallel.xml).

8.3 FailedTestRunner — Re-run Mechanism

When any scenario fails, both TestRunner and ParallelRunner record its location to target/failed_scenarios.txt via the rerun: Cucumber plugin. FailedTestRunner reads this file as its feature source, re-executing only the failed scenarios. This dramatically reduces re-run cycle time when only a small subset of tests fail.

8.4 Parallel Execution Configuration Summary

Configuration Item	**Value / Setting**
Parallel DataProvider	@DataProvider(parallel = true) in ParallelRunner
Thread Isolation Mechanism	ThreadLocal<WebDriver> in DriverFactory

Configuration Item	Value / Setting
Thread-per-scenario	Yes — each Cucumber scenario gets its own thread and WebDriver instance
Failed test re-run	rerun: plugin writes to target/failed_scenarios.txt
TestNG XML (parallel)	testng-parallel.xml configured for parallel mode to align with DataProvider parallelism

8.5 ThreadLocal WebDriver Lifecycle

The ThreadLocal<WebDriver> pattern guarantees that each parallel scenario receives a dedicated, isolated browser instance. The lifecycle is as follows:

- @Before Hook: DriverFactory.initDriver(browser) creates a new WebDriver for the current thread. ConfigReader supplies the browser name from config.properties.
- During Execution: All Page Object classes call DriverFactory.getDriver() to retrieve the thread-local instance. No driver reference is ever shared between scenarios.
- @After Hook: DriverFactory.quitDriver() calls driver.quit() and removes the ThreadLocal reference, preventing memory leaks between scenarios.

9. Reporting and Logging

The framework provides a comprehensive, multi-layered reporting and logging strategy. Every test run produces interactive Allure reports, Cucumber HTML and JSON reports, a TestNG summary, and a structured Log4j2 log file. Screenshots are automatically embedded in reports on UI test failure.

9.1 Allure Reporting

Allure is integrated into both ui-automation and api-automation modules via the allure-cucumber7-jvm adapter (version 2.29.0). The adapter automatically collects Cucumber scenario data — including step status, duration, tags, and failure details — and writes result files to target/allure-results/ after each run.

To generate and serve the interactive Allure HTML report after a test run:

```
allure serve target/allure-results
```

Allure reports provide:

- Per-scenario step-by-step breakdown with pass/fail/skip status.
- Execution timeline showing scenario durations.
- Embedded screenshots on failure directly within the scenario report.
- Tag-based filtering and suite grouping.
- Historical trend comparison across runs (when using CI integration).

9.2 Cucumber HTML and JSON Reports

Both TestRunner and ParallelRunner configure the following Cucumber plugins that generate additional report artifacts:

Plugin	**Output Location**	**Purpose**
pretty	Console / stdout	Prints formatted step-by-step execution output to the terminal during the run.
html:...	target/cucumber-reports/report.html	Self-contained HTML report with step details, scenario results, and durations.
json:...	target/cucumber-reports/report.json	Machine-readable JSON report for CI tool integration (Jenkins, GitLab).

Plugin	Output Location	Purpose
rerun:...	target/failed_scenarios.txt	Records failed scenario locations for FailedTestRunner re-execution.

9.3 Screenshot Capture on Failure

The Hooks class in the ui-automation module captures a full-page screenshot automatically on every UI scenario failure. The screenshot pipeline works as follows:

- ScreenshotUtil.capture(driver) invokes Selenium's TakesScreenshot interface and returns the screenshot as a byte[] array.
- The byte[] is passed to scenario.attach(screenshot, "image/png", "Failure Screenshot") inside the @After hook.
- Cucumber embeds the screenshot directly into both the HTML report and the Allure result file.
- This makes every failure self-contained and investigation does not require access to the test machine.

9.4 Log4j2 Logging

Log4j2 is configured in ui-automation/src/test/resources/log4j2.xml. The common module's LogsFactory class provides Logger instances to any class in both modules using a single, centralized logging configuration.

Logging is applied consistently across:

- Driver initialization and configuration loading.
- Each Cucumber step execution entry and exit.
- Page object actions and element interactions.
- API request dispatch and response receipt.
- All error conditions and caught exceptions.

Log output is directed to target/logs/automation.log, enabling post-run log review without console access.

9.5 Complete Reporting Artifact Summary

Artifact	Location	Contents
Allure Report	target/allure-results/	Interactive report with step timeline, attachments, and embedded screenshots.

Artifact	Location	Contents
Cucumber HTML	target/cucumber-reports/report.html	Human-readable pass/fail report with step details and scenario summary.
Cucumber JSON	target/cucumber-reports/report.json	Machine-readable report for CI pipeline integration.
TestNG Report	test-output/index.html	TestNG built-in HTML report with test suite summary and configuration.
Log File	target/logs/automation.log	Full structured execution log from Log4j2 file appender.
Failed Scenarios	target/failed_scenarios.txt	Rerun file for FailedTestRunner containing failed scenario locations.

10. Technology Stack Summary

The table below summarizes all technologies, libraries, and tools used in this framework, along with their versions and roles across each module.

Technology / Library	Version	Module(s)	Role / Purpose
Java (JDK)	21	All	Primary programming language for all framework source code.
Maven	3.x (latest)	All	Build tool, dependency management, and multi-module project orchestration.
Selenium WebDriver	4.41.0	ui-automation	Browser automation library for all UI test interactions.
WebDriverManager	6.3.3	ui-automation	Automatic browser driver binary resolution and management.
Cucumber (Java + TestNG)	7.34.3	ui, api	BDD framework mapping Gherkin feature file scenarios to Java step definitions.
TestNG	7.12.0	ui, api	Test execution engine providing parallel DataProvider, assertions, and suite management.
RestAssured	5.5.7	api-automation	HTTP client for constructing and executing REST API test requests and validations.
RestAssured JSON Schema Validator	5.5.7	api-automation	JSON schema validation against stored schema files for contract testing.
Log4j2 (API + Core + SLF4J bridge)	2.25.3	All (via common)	Structured application logging to console and file via LogsFactory.
Allure Cucumber7 JVM	2.29.0	ui, api	Rich interactive test reporting with Allure framework integration.
org.json	20251224	api-automation	Lightweight JSON object construction and parsing.

Technology / Library	Version	Module(s)	Role / Purpose
SLF4J Simple	2.0.9	All (parent)	SLF4J binding to suppress third-party framework logging noise.
Git	—	All	Version control. Branching strategy: main, dev, feature branches per test case.
IntelliJ IDEA / Eclipse	—	—	IDE with Cucumber and Gherkin plugin support for feature file editing.

10.1 Technology Dependency Map

The table below groups the technology stack by module and layer to show how tools relate to each other within the framework:

Module / Layer	Primary Technology	Supporting Libraries
Parent POM	Maven 3.x	Java 21 (compiler target), SLF4J Simple
common	Log4j2 2.25.3	SLF4J bridge, LogsFactory facade
ui-automation — Driver	Selenium WebDriver 4.41.0	WebDriverManager 6.3.3, ChromeDriver, EdgeDriver
ui-automation — BDD	Cucumber 7.34.3 + TestNG 7.12.0	Gherkin .feature files, AbstractTestNGCucumberTests
ui-automation — Reporting	Allure 2.29.0 + Cucumber HTML	allure-cucumber7-jvm, Log4j2 (via common)
api-automation — HTTP	RestAssured 5.5.7	JSON Schema Validator 5.5.7, org.json 20251224
api-automation — Data	JavaFaker 1.0.2	Jackson 2.17.0, UserDataBuilder, User POJO
api-automation — BDD	Cucumber 7.34.3 + TestNG 7.12.0	ProductRunner, feature files, Step Definitions
api-automation — Reporting	Allure 2.29.0	Cucumber JSON, Log4j2 (via common)

10.2 Build and Execution Commands

All modules are built and executed through Maven. The following commands cover the most common operations:

Run all tests (sequential):

```
mvn clean test
```

Run UI tests with parallel execution:

```
mvn clean test -pl ui-automation -Dsurefire.suiteXmlFiles=testng-parallel.xml
```

Re-run only failed UI scenarios:

```
mvn test -pl ui-automation -Dsurefire.suiteXmlFiles=testng-rerun.xml
```

Generate and open Allure report:

```
allure serve target/allure-results
```

Run API tests only:

```
mvn clean test -pl api-automation
```